

Support Casebook

A working notebook of six diagnosed support cases, each documented in the same five-section shape: symptom, diagnosis, root cause, fix, outcome.

Who and what

This casebook collects the diagnostic notes from a working support lab. It is not a tutorial. It is the kind of writeup a support engineer leaves behind so that the next person to see the same ticket can resolve it from the writeup alone.

The cases span:

- authentication failures (JWT)
- database state drift (Postgres migrations)
- container-network footguns (loopback bind)
- integration security (HMAC over raw bytes vs re-serialized JSON)
- performance regressions (N+1 query patterns)
- Linux-native DNS quirks (Alpine + musl + ndots)
- TLS handshake failures (incomplete certificate chain)

The repository is reproducible end-to-end: every case has a `reproduce.sh` that puts the platform into the broken state and exercises the failure. CI runs every reproduction and diffs the output.

Why configuration drift escapes tests: the symmetry trap

A pattern that keeps reappearing in this casebook deserves its own name, because once it has one the lessons stop being case-by-case.

Symmetry trap. A test cannot disagree with the production code about a value when both sides read that value from the same source of truth. The test passes against any setting the source happens to hold; the production code accepts any token, request, or configuration the test happens to mint. The bug only surfaces when something *outside* the symmetric pair — a customer's IdP, a client's TLS stack, a sender's JSON library, a deploy that lands a different config in one environment than another — disagrees with what the test and the production code agree on. By that point the bug has shipped.

The trap shows up in three cases here:

- Case 01 (JWT audience): the test minted tokens with `JWT_AUDIENCE` from the same env file the verifier read; the verifier rejecting tokens with a different audience was untested.
- Case 04 (HMAC over JSON): the test signed payloads using the same `json.dumps(separator=...)` call as the verifier; the verifier signing different bytes than a real sender was untested.
- Case 07 (TLS chain): the deploy was tested by curl-ing from an operator laptop that already had the new intermediate cached; the from-cold-trust-store path was untested.

Recipe for breaking the symmetry. The test must use a *different* source of truth from the code. Concretely: a fixture with a hardcoded counterexample value, or a clean environment that excludes whatever the code has ambient access to. The pinning tests in `tests/test_auth.py`, `tests/test_cert_chain.py`, and `tests/test_orders.py` each do this in their own way — the audience values, the cert files, the query counts are constants chosen to *not* match anything the code reads at runtime. If the code regresses, those constants disagree with the new behavior and the test fails immediately.

Two-second check. Look at any test for code that touches configuration. Ask: where does the test get its expected values from? If it gets them from the same place the code does, the test cannot catch configuration drift. The fix is always the same: hardcode the test's expectations, or pull them from a fixture that doesn't know about the code's runtime environment.

Case 01 — 401 from JWT audience-claim mismatch

Symptom. customer's tokens are signature-valid and unexpired but get 401 from /me .

Diagnosis. `curl -v` shows the 401 originates in the application, not a proxy. Decoded the token without verification: signature matches when re-encoded with the dev secret. Read claims: `aud="api-staging"` . Verifier configured with `JWT_AUDIENCE=api` . Claim mismatch.

Root cause. customer's IdP issues `aud="api-staging"` for staging; the verifier on the staging environment was deployed with a single audience value. Environmental config drift.

Fix. customer-side workaround mints with `aud="api"` . Engineering fix: change the verifier to accept a list of audiences (`JWT_AUDIENCES`), ship per-environment.

Outcome. ticket resolved in <30 minutes once request ID was supplied. Engineering work prevents the next instance from filing.

Case 02 — Postgres “relation `audit_log` does not exist” after partial migration

Symptom. /audit returns 500 on staging only; production fine; nothing has changed in the schema “from the customer's side.”

Diagnosis. error message is its own diagnosis. `SELECT MAX(version) FROM schema_migrations` returns 1 — migration 002 was never applied here. `\dt` corroborates: the `audit_log` table 002 creates is missing. /healthz?check=schema returns 503 schema behind: expected=2 actual=1 .

Root cause. migration 002 was not applied on this node. The `schema_migrations` registry makes this a one-query answer.

Fix. apply 002 (the runner inserts the registry row on success). The schema-aware healthcheck and the deploy-pipeline gate that uses it are already shipped; wire the gate into the customer's readiness probe so a node behind on migrations never receives traffic.

Outcome. the missing piece worth shipping is the deploy-pipeline gate — the next instance of this should fail readiness rather than serve 500s.

Case 03 — Container reachable from inside, refused from host (bind 127.0.0.1)

Symptom. in-container healthcheck passes, host curl on the published port returns connection refused.

Diagnosis. `docker compose ps` shows the port published. `ss -tulpn inside the container` shows the application listening on `127.0.0.1:8000` , not `0.0.0.0:8000` . Port is forwarded fine; the application itself is bound to the container's loopback, which is not reachable from outside the container.

Root cause. `uvicorn` started with `--host 127.0.0.1` . Container loopback ≠ host loopback.

Fix. bind `0.0.0.0` . Image default should make the wrong thing impossible.

Outcome. classical container-networking footgun. Worth fixing in the reference image so the next self-onboarding customer doesn't bounce.

Case 04 — Webhook HMAC fails after JSON re-serialization

Symptom. every event from a previously-working sender is rejected with `bad signature` .

Diagnosis. wrong first hypothesis was clock skew on the timestamp header — disproved by replaying with the sender's exact timestamp. Second wrong hypothesis was wrong secret — disproved by manual HMAC computation. Third hypothesis stuck: receiver computes HMAC over `json.dumps(parsed, separators=(",", ":"))` instead of the raw request bytes. Sender uses different JSON whitespace; bytes diverge; HMACs diverge. The `body_len=` field on the log line confirms the byte-count delta.

Root cause. `await request.json()` runs before the HMAC check; HMAC is computed over a re-serialized form. Verifier is signing different bytes than the sender.

Fix. capture `await request.body()` first, verify HMAC over the raw bytes, then parse. The “verify before parse” rule, applied universally to signed payloads.

Outcome. once shipped, the same payload+headers verify cleanly. Pinning test added to fail if anyone reintroduces parse-before-verify.

Case 05 — `/orders` p99 spikes from 50ms to 1.4s under load

Symptom. single request fast, batch slow. Customer's APM shows p99 1.4s on `/orders` .

Diagnosis. API logs `queries=N+1` per `/orders?limit=N` . `pg_stat_statements` snapshot confirms the per-row customer query is being called 100× more often than the bulk-orders query. EXPLAIN ANALYZE of the per-row query is fast; the problem is round-trip count, not query plan.

Root cause. `N+1`. Endpoint loops over orders and queries customers per row.

Fix. rewrite as a two-query pattern with `customers WHERE id = ANY(%s)` . Add a regression test that asserts `queries <= 2` regardless of `limit` .

Outcome. post-fix latency is per-query round-trip × 2 instead of × `limit+1` . The query-count assertion (not the absolute timing) is the part that prevents the regression from re-landing.

Case 06 — Intermittent DNS in Alpine container

Symptom. worker container resolves a Compose service name “about a third of the time.”

Diagnosis. ran the lookup back-to-back five times — same result every time, so the failure is not literally random; it is config-state-dependent and looks random from outside. `/etc/resolv.conf` shows `127.0.0.11` (Docker embedded DNS) and unusual `ndots / search` config. `getent hosts api` (musl resolver path) and `dig api` diverge depending on `resolv.conf` state and trailing-dot. FQDN with trailing dot is reliable.

Root cause. musl resolver semantics differ from glibc; combined with Docker embedded DNS expectations and `resolv.conf` state, short-name lookups can miss.

Fix. customer-side, use `api.` (FQDN). Image-side, switch off Alpine to a glibc base.

Outcome. durable fix is the image change. Doc + sample-image update prevents the next ticket.

Case 07 — TLS handshake fails: server presents leaf only

Symptom. “half of our clients” fail with unable to get local issuer certificate ; the other half succeed against the same endpoint.

Diagnosis. `openssl s_client -showcerts -CAfile <client root bundle>` against the endpoint shows a `Certificate chain` block with one cert (the leaf). Verify return code 21 / errno 20. Cold clients fail; clients with the intermediate already cached from an unrelated session validate locally and succeed — that’s the “half work” framing.

Root cause. server’s loaded cert file contains only the leaf, not leaf + intermediate. Surfaces after CA-vendor changes whose issuance scripts emit leaf and intermediate as separate files.

Fix. `cat leaf.pem intermediate.pem > fullchain.pem` , point the server config at the concatenated file, reload. Pipeline-side: enforce a chain-completeness check on issuance.

Outcome. the workaround is two minutes. The pipeline check is what stops the next CA rotation from doing the same thing.

What this casebook is signaling

A support engineer who can:

- separate workaround from fix and write the customer in workaround voice while writing engineering in fix voice
- form a hypothesis, pick the cheapest test that could disprove it, run that one first, and document the dead hypotheses as well as the live one
- read evidence from `ss` , `dig` , `EXPLAIN ANALYZE` , `pg_stat_statements` , and structured request logs without ceremony
- write down the writeup that turns a 30-minute ticket into a 30-second ticket the next time the same customer or a different customer files it